

《AArch64 Type-1 Hypervisor 调试笔记：由 1bit 引发的 EL2->EL1 架构错位 BUG》

By Jia

在 AArch64 架构下，实现一个 Type-1 Hypervisor，借助多个 AI 协同（GPT 5.2 / Claude Sonnet 4.5 + Claude Code）确实效率提高很多，以往需要几个人同时做的事情，现在基本一个人就能完成，可以省出大把时间把精力放在构思与细节上。关注细节是因为之前用 AI 辅助编程就踩过一些坑，大多是因不同上下文环境产生的 race condition 与 deadlock。所以我觉得当下的 AI 在复杂场景下的能力边界是需要讨论的，尤其是存在 3 个以上机制的交叉点，同时牵扯到并发，锁和 Arch 相关的场景。以我个人经验，此类问题现阶段无法避免，但可以努力减少出问题的概率，方法就是在开始之前尽最大努力给 AI 介绍自己的配置环境，运行场景，并让它重复这些信息，然后多次提问并纠正，确保它已足够理解。但即便这样，该来的还是会来，该还的还是会还。比如这次遇到的 BUG，尝试多轮修正无果后，我将模型切换到了 Claude Opus 4.5 它给我的建议是：不要用 qemu，换真实环境测试。虽然没有直接定位 BUG，但也算提醒我与运行环境有关，所幸因切换环境过于繁琐，我坚持在当前环境继续深挖，否则这个 BUG 很可能被掩盖。

前期我已详细给出了 qemu 的参数和运行环境（Type-1 Hypervisor 以 EL2 直接运行在“裸机”上），并自己先写基础代码，然后再让 AI 在此基础上修改，扩充。可以说步步为营，但还是出了这个“有趣”的 BUG。事后总结：不要假设 AI 对已有语料的运用，比如我这次我就有个假设的前提，对于 x86_64 和 AArch64 这样架构相关的细节问题早已进入 AI 语料，它可以根据你提供的场景，精准的运用推理给出标准答案，可结果并非如此。其次就是“主导”问题，如果让 AI 主导，自己跟随，一旦 AI 开始出现幻觉，便无从选择，寸步难行。以截止到今天 AI 的能力，它只配给你打工，而不是你给他打工（前提是你很清楚自己在做什么，并有相关领域的背景知识，可以分辨出 AI 回答中的问题）。其实做到这点挺难的，在这篇笔记中的前半段，我都是在跟随 AI，因为 AI 太便利了，它总能给你一个答案，然后这个球就给到了你，你是选择相信，沿着它的思路走，还是选择不信任...时间都在验证中流失...事后总结也提醒到我，一些传统习惯还是有必要保留的，比如看手册。这次如果没有手册为依据很难想象还要走多少弯路。

这次 BUG 的表象是启动初始阶段需使用 eret 指令从 EL2->EL1。当切换后，任意指令都会触发一个“Synchronous exception”。相关实现过程参考 2025 最新版的《Arm® Architecture Reference Manual》手册（感谢 lhs 及时提供）：

依据手册描述，当执行 eret 指令时，AArch64 硬件自动完成以下动作：

- (1) 将 ELR_EL2 的值加载到 EL1 的 PC。
- (2) 将 SPSR_EL2 的值复制到 PSTATE。
- (3) 完成当前 EL2->EL1 异常级别切换。

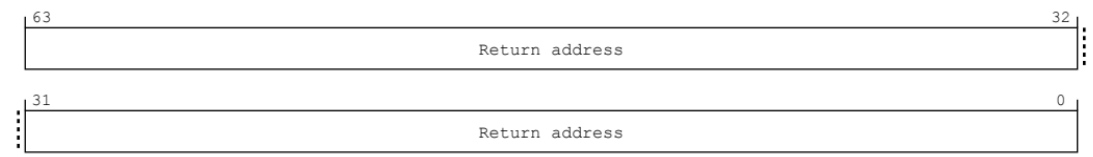
(4) 更新 EL1 stack pointer.

注: PSTATE 是 AArch64 的叫法; 为了兼容 AArch32, 在 LLDB 中仍将叫 cspr, 想要查看 PSTATE 内容, 指定的仍是 cspr, 只不过换了个叫法。

接下来就是依据手册按步骤设置相关寄存器

设置 ELR_EL2:

当 AArch64 模式下 EL2->EL1 的时候 ELR_EL2 寄存器必须被设置, 这是一个 64bit 寄存器



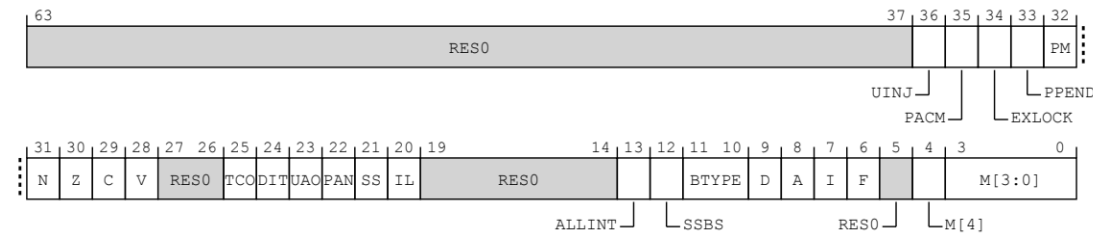
根据手册描述, 在我当前场景下, 直接写切换到 EL1 后需要执行的指令, 也就是入口函数地址:

```
(lldb) reg read elr_el2
ELR_EL2 = 0x0000000040200304 hyper-vm`el1_entry at main.rs:183
```

```
(lldb) x/6i $elr_el2
0x40200304: 0xd10043ff unknown sub sp, sp, #0x10
0x40200308: 0xd5384248 unknown mrs x8, CurrentEL
0x4020030c: 0xf90007e8 unknown str x8, [sp, #0x8]
0x40200310: 0xf94007e8 unknown ldr x8, [sp, #0x8]
0x40200314: 0xd3420d08 unknown ubfx x8, x8, #2, #2
0x40200318: 0xf90003e8 unknown str x8, [sp]
```

设置 SPSR_EL2:

当 AArch64 模式下 EL2->EL1 的时候 SPSR_EL2 寄存器必须被设置, 这是一个 64bit 寄存器



其中 M[3:0] 相关 bit 属性如下：

M[3:0]	Meaning
0b0000	EL0.
0b0100	EL1 with SP_EL0 (EL1t).
0b0101	EL1 with SP_EL1 (EL1h).
0b1000	EL2 with SP_EL0 (EL2t).
0b1001	EL2 with SP_EL2 (EL2h).

在当前环境，只讨论 EL1h 即 0b0101 模式：

- (1) M[3:2] 当在 EL2 中执行异常返回操作时，该值复制 PSTATE.EL。
- (2) M[1] 未使用，对于所有非保留值均为 0。
- (3) M[0] 当在 EL2 中执行异常返回操作时，复制到 PSTATE.SP。

关于 M[4] bit 说明：

M[4], bit [4]

Execution state. Set to 0b0, the value of PSTATE.nRW, on taking an exception to EL2 from AArch64 state, and copied to PSTATE.nRW on executing an exception return operation in EL2.

uction Class
registers

M[4]	Meaning
0b0	AArch64 execution state.

EL1h 与 EL1t:

这里的 "h"与 "t" 被 AArch64 手册解释为：“handler” 和 “thread”
对于我们当前环境必须设置为 handler，也就是必须为 EL1 分配并配置栈空间，原因是从 EL2 返回到 EL1 时，EL1 必须有自己的栈；而 thread 模式则表明为共享栈空，如 EL0 和

EL1 之间共享，但在当前“裸机”环境下，根本没有 EL0 配置，而且这种共享这多见于 bootloader 场景下，这里不在讨论。

对 SPSR_EL2 最终设置

```
(lldb) reg read spsr_el2 --format bin
```

SPSR_EL2 =

[illegible]

- (1) N/Z/C/V: 相关条件标志均无设置
- (2) D/A/I/F: 已设置屏蔽 IRQ/FIQ/Debug/SError 所有中断和硬件错误
- (3) nRW(M[4:0]): 当前架构位 AArch64 **execution state** (**注意这里**)
- (4) EL1(M:[3:2]): 已设置返回至 EL1
- (5) SPSel(M[0]): 已设置使用 SP_EL1

也就是说当前设置符合手册描述的 **M[3:0] = 0101 = EL1 with SP EL1(EL1h)**

当 SPSR_EL2 设置为 EL1h, 执行 eret 指令切换到 EL1h 后, 硬件会立即开始使用 SP_EL1 来压栈, 如果没有提前设置 SP_EL1, 那么第一条指令就会写入非法地址 (如 0x0)。导致 Data Abort 或直接死机。**所以我当时一直怀疑是我 EL1 stack 或 EL1 stack pointer 设置的有问题。**

```
(lldb) reg read elr el2
```

ELR_EL2 = 0x00000000402002f0 hyper-vmml_el1_entry at main.rs:180

```
(lldb) reg read sp el1
```

SP_EL1 = 0x0000000040203000 hyper-vmx`_stack

```
(lldb) memory read $sp el1
```

```
0x40203000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

```
0x40203010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

(lldb)

通过观察，我的 ELR_EL2 值设置正确，在 LLDB 中还未继续做跟踪操作 (si/s/n/c) 时，0x00000000402002f0 这个地址就是我的 el1 entry() 入口地址。

(lldb) x/3i \$pc

```
-> 0x40200414: 0xd69f03e0    unknown    eret
```

```
0x40200418: 0x14000001   unknown    b
0x4020041c          ; <+260> [inlined] hyper_vmm::cpu_idle + 4 at
main.rs:201:13
0x4020041c: 0xd503205f   unknown    wfe
(lldb)
```

此时我在 `eret` 指令处设置断点并在 LLDB 中执行 `si` 命令：

```
(lldb) si
Process 1 stopped
* thread #1, stop reason = instruction step into
  frame #0: 0x00000000402002f0 hyper_vmm`el1_entry at main.rs:180
  177
  178  #[inline(never)]
  179  #[no_mangle]
-> 180  extern "C" fn el1_entry() -> !
  181  {
  182      let _el = read_current_el();
  183
```

到这里看似一切正常，已经看到了 EL1 入口函数 `el1_entry` 了，再次执行 `si` 命令：

```
(lldb) si
Process 1 stopped
* thread #1, stop reason = instruction step into
  frame #0: 0x0000000000000200
-> 0x200: udf    #0x0
    0x204: udf    #0x0
    0x208: udf    #0x0
    0x20c: udf    #0x0
(lldb)
```

到这里就比较奇怪了，明明显示的是 `el1_entry` 中的获取 `CurrentEL` 寄存器的代码，为什么是 `udf` 呢？`udf` 即“未定义指令异常”（Undefined Instruction exception），这里的 `udf` 被视为一种“Synchronous exceptions”（同步异常），何为同步？与之相对的异步异常又是什么？在 AARCH64 手册的 D1.4.1.3 有如下定义：

D1.4.1.3 Synchronous and asynchronous exceptions

If all of the following apply, an exception is *synchronous*:

- The exception is generated as a result of direct execution or attempted execution of an instruction.
- The preferred exception return address has an architecturally defined relationship with the instruction that caused the exception.
- The exception is precise.

An exception is *asynchronous* if it is not synchronous.

Asynchronous exceptions taken to AArch64 state are also known as interrupts.

简而言之就是包括但不限于 load/store 指令引发的异常都叫同步异常，而异步异常则是指 IRQ/FIQ/SError 这些由外设或 CPU CORE 内部错误导致的异步异常，在 AArch64 下统称为 Interrupts（中断）。每个异常级别（EL0/EL1/EL2/EL3）都有自己的 Vector Base Address Register (VBAR_ELx)，指向一个 256 字节的异常向量表。该表包含 16 个入口，每个 16 字节（4 条指令）；在手册的 D1.4.1.6 Exception vectors 描述如下：

D1.4.1.6 Exception vectors

When an exception is taken to an Exception level that is using AArch64, execution starts at the *exception vector*.

The memory at the exception vector for an exception is expected to contain the handler code that corresponds to that exception category.

The vector base address for an Exception level, ELx, is defined by the Vector Base Address Register, VBAR_ELx.

Each exception category has an exception vector at a fixed offset from the vector base address.

An exception taken to AArch64 is categorized for the purpose of assigning an exception vector based on the following information:

- Whether the exception is one of the following:
 - A synchronous exception, excluding synchronous External aborts.
 - A synchronous External abort.
 - An SError exception, including vSError and delegated SError exceptions.
 - An IRQ or vIRQ interrupt.
 - An FIQ or vFIQ interrupt.
- Information about the Exception level that the exception came from.
- Information about the SP in use.
- The state of the register file.

For information on synchronous exceptions, see [Synchronous exception types](#). For information on asynchronous exceptions, see [Asynchronous exception types](#).

根据手册中对 exception vector table 的定义，每个入口都对应了相应的偏移，根据异常类型、来源 EL、SP 状态等，跳转到 VBAR_ELx + offset 的不同入口，这个 offset 是固定，每类占用 16 个字节，通常放一条 b xxx_handler 指令。具体固定偏移如下：

Exception taken from	Offset for exception type		
	Synchronous exceptions, excluding External aborts	IRQ and vIRQ	FIQ and vFIQ
Current Exception level with SP_ELO.	0x000	0x080	0x100
Current Exception level with SP_ELx, x > 0.	0x200	0x280	0x300
Lower Exception level, where the implemented level immediately lower than the target level is using AArch64.	0x400	0x480	0x500

Exception taken from	Offset for exception type		
	Synchronous exceptions, excluding External aborts	IRQ and vIRQ	FIQ and vFIQ
Lower Exception level, where the implemented level immediately lower than the target level is using AArch32.	0x600	0x680	0x700

Exception taken from	Offset for exception type		
	Synchronous External aborts, EASE == 0	Synchronous External aborts, EASE == 1	SError, vSError, and delegated SError
Current Exception level with SP_ELO.	0x000	0x180	0x180
Current Exception level with SP_ELx, x > 0.	0x200	0x380	0x380
Lower Exception level, where the implemented level immediately lower than the target level is using AArch64.	0x400	0x580	0x580
Lower Exception level, where the implemented level immediately lower than the target level is using AArch32.	0x600	0x780	0x780

通过产生 udf 处的偏移 0x200 对应到手册，我当前情况完全符合手册中所说的 SP_ELx > 0 的情况，此时我把全部代码和异常情况同时喂给 GPT 5.2 与 Claude Sonnet 4.5 他们分别给出了几种可能性，其中看着最靠谱的就是说我因为没有在 EL1 实现 exception vectors table 而导致的，为了验证到底为什么会有 udf，而且 udf 是如何产生的，是 Instruction exception 还是 Data Abort 引起的，只要实现 exception vectors table 就可以在 Synchronous offset 0x200 处捕获该异常，从而根据 ESR_EL1/FAR_EL1 寄存器来定位 BUG。（注意：到现在为止我仍然坚定认为已经切换到了 EL1，理由是 LLDB 打印出的 SP_EL1 以及\$PC 处的汇编代码）

根据手册规定，在实现 exception vector table 时必须是以 2K 进行对齐。我的 el1_vectors.S 中的相关代码：

```
.section .text.el1_vectors, "ax"
.align 11
.global el1_vector_base
```

使用 objdump 查看

Sections:						
Idx	Name	Size	VMA	LMA	File off	Algn
0	.text	00000d4c	0000000040200000	0000000040200000	00010000	2**11
CONTENTS, ALLOC, LOAD, READONLY, CODE						

引用与设置 VBAR_EL1 相关代码如下:

```
extern "C" {
    static el1_vector_base: u8;
}
```

```
let vbar_addr = &el1_vector_base as *const _ as u64;
write_vbar_el1(vbar_addr);
```

当再次执行 eret 指令从 EL2->EL1 后查看将要执行的下一条指令, 到现在为止一切正常:

```
(lldb) x/6i $pc
-> 0x40200304: 0xd10043ff  unknown  sub    sp, sp, #0x10
    0x40200308: 0xd5384248  unknown  mrs    x8, CurrentEL
    0x4020030c: 0xf90007e8  unknown  str    x8, [sp, #0x8]
    0x40200310: 0xf94007e8  unknown  ldr    x8, [sp, #0x8]
    0x40200314: 0xd3420d08  unknown  ubfx   x8, x8, #2, #2
    0x40200318: 0xf90003e8  unknown  str    x8, [sp]
(lldb)
```

当前 PC 指向的正是 el1_entry 入口处的代码, 我将当前执行环境贴给 GPT 5.2 与 Claude Sonnet 4.5 它们一致认为已经切换到了 EL1 中, 再根据手册中的描述, 我认为接下来的 si 指令应该会跳转到我定义的 exception vector 处。可惜, 并没有! 依然是执行到 el1_entry 后再次 si 触发异常, udf 的地址依然是 0x200 处, 并没有进入到我的预先设定的 exception vector 中。到这一步我同样认为问题出在没有正确设置 VBAR_EL1, 而导致 exception vector table 的 base 为 0, 所以 0 + 0x200 处不可能有对应的手册修改代码, 在同步异常的偏移 0x200 处, 添加如下代码。

```
(lldb) x/4i "vbar_addr + 0x200"
    0x40200a00: 0xa9bf07e0  unknown  stp    x0, x1, [sp, #-0x10]!
    0x40200a04: 0xd538c000  unknown  mrs    x0, VBAR_EL1
    0x40200a08: 0xd5385201  unknown  mrs    x1, ESR_EL1
```



```
0x40200a0c: 0x14000000    unknown    b
0x40200a0c          ; <+12>
(lldb)
```

但无论如何，每次 `eret` 执行跳转到 `el1_entry()` 入口后，无论如何都无法继续执行，哪怕是 `nop` 这种指令，都会触发 `udf`；在这个上下文中同样无法进入 `VBAR_EL1` 指定的偏移。

在这之期间，我尝试过使用 Claude Code；或从 0 开始把代码分批输入给 AI (GPT 5.2 / Sonnet 4.5)，更换提示词，赋予多个不同专家身份，它们给出的答案也是五花八门，但看着就不太靠谱，有说我因为没有设置 `VTTBR_EL2` 的，说我没有在 `SPSR_EL2` 允许 `Debug bit` 的，说因为 `Stage-2` 错误的，我说我当前 `MMU` 都是 `off` 的，此时 AI 给出的回答让我开启 `MMU`，接着就是 `VM bit` 配置问题等等...这些看过手册对 `ARMv8` 架构稍微了解些的就知道不太靠谱，但什么叫有病乱投医？为了赌一把，AI 所列出的可能性我都对照着手册该开启的开启该使能的使能，但问题依旧，此时已头晕脑胀，决定不在跟随 AI，而是反过来让它跟着我的步调走。我开始依据当前上下文精读手册，针对每一步的调试，找出所有相关寄存器，观察所有 `bit`，列出所有条件，确认每一步手册中列出的所有相关寄存器 `bit` 位，所有状态都无误后再进行下一步，直到 `eret` 指令切换时，我能确定正确内容：

- (1) `SPSR_EL2` 设置正确
- (2) `ELR_EL2` 设置正确
- (3) `VBAR_EL2` 设置正确
- (4) `SP_EL1` 设置正确
- (5) Exception vector table 设置正确 (2K 对齐 / 偏移正确)
- (6) `EL1h stack buffer` 设置正确

因为上述的正确性，所以执行 `eret` 后跳转到了 `el1_entry` 处，同时 `PC` 指向了正确的位置。因 exception vector table 无法捕获 `0x200` 处的异常，导致 `mrs x1, ESR_EL1` 无法执行，我这里尝试用 LLDB 直接查看 `ESR_EL1` 的值：

```
(lldb) reg read ESR_EL1
ESR_EL1 = 0x0000000000000000
```

没有发现任何异常。观察 `PSTATE` (非物理寄存器)，按照手册说法，低位由 `cpsr` 寄存器指定：

```
(lldb) reg read cpsr
cpsr = 0x000003c9    <- 蛛丝马迹
```

(lldb)

开篇介绍过 PSTATE 也就是这里的 `cpsr`，在执行 `eret` 指令时内容来自 `SPSR_EL2`。再次观察 `SPSR_EL2`

```
(lldb) reg read SPSR_EL2
```

```
SPSR_EL2 = 0x00000000000003c9
```

```
(lldb) reg read SPSR_EL2 --format bin
```

SPSR_EL2 =

[illegible]

这里的 **M[3:0]**与切换前的对不上，我之前写入的是 **0101** 执行完 **eret** 后怎么变成了

1001? 根据手册提示 1001 对应的是 **EL2 with SP_EL2(EL2h)**，抓住这一点查看 EL2 相关寄存器

```
(lldb) reg read VBAR EL2 ELR EL2
```

VBAR_EL2 = 0x0000000000000000

ELR EL2 = 0x0000000000000200

(lldb)

这就有意思了 ELR_EL2 正好是 0x200 即产生异常处，而它的基地址 VBAR_EL2 为 0，这样来说 udf 在 exception vectors table 入口由 $VBAR_ELR + ELR_EL2 = base + offset = 0 + 0x200$ 这就都对上了，这个同步异常产生在 EL2 ？！**那在执行 eret 后能够正确跳转到 el1_entry 同时 PC 能够正确指向入口代码又作何解释呢？**原来为 EL1 准备 exception vector 是为了能够捕获更多异常信息，既然现在系统提示异常发生在 EL2，那么直接查看 ESR_EL2 即可：

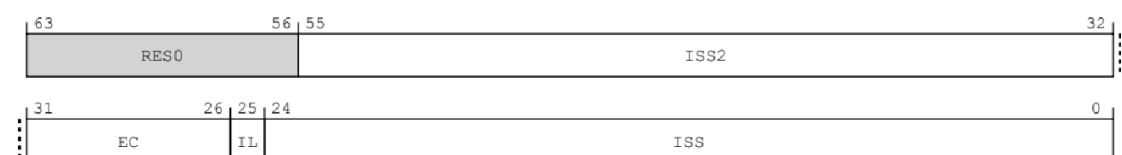
```
(lldb) reg read ESR_EL2 --format bin
```

ESR EL2 =

[illegible]

(lldb)

在 ESR_EL2 有置位，难怪 ESR_EL1 没有任何异常提示，原来问题都出在 EL2。查看手册中关于 ESR_EL2 定义：



当前的 ESR_EL2 第 25 bit 置位即 IL (Instruction Length)

IL	Meaning
0b0	16-bit instruction trapped.
0b1	32-bit instruction trapped. This value is also used when the exception is one of the following: • An SError exception. • An Instruction Abort exception. • A PC alignment fault exception. • An SP alignment fault exception. • A Data Abort exception for which the value of the ISV bit is 0. • An Illegal Execution state exception. • Any debug exception except for Breakpoint instruction exceptions. For Breakpoint instruction exceptions, this bit has its standard meaning: • 0b0: 16-bit T32 BKPT instruction. • 0b1: 32-bit A32 BKPT instruction or A64 BRK instruction. • An exception reported using EC value 0b000000.

这里看到的 IL 只代表了指令长度，是无法确定到底是以上哪种异常引起的，此时的 EC (Exception Class) 即第 EC[31:26] 没有给出任何信息，这个范围太大了。

我来回调试了 3 次后最终发现，在 eret 指令执行后还一切正常，如果要得到有效的异常信息，必须执行一条 si 命令且后续不能再有任何动作，此时马上观察相关寄存器才可以看到 ESR_EL2 的 EC 值，如果此时再执行诸如 si/s/n/c 命令那么相关 bit 都会被清空，最终只在 SPSR_EL2 的 M[3:0]与 ESR_EL2 的[25]中进行置位，其它 bit 均已清空。

Process 1 stopped

* thread #1, stop reason = instruction step into

frame #0: 0x00000000402002f0 hyper-vm`el1_entry at main.rs:180

177

178 #[inline(never)]

179 #[no_mangle]

-> 180 extern "C" fn el1_entry() -> !

181 {

182 let _el = read_current_el();

183

(lldb) reg read SPSR_EL2 ESR_EL2

SPSR_EL2 = 0x000000000000003c5

ESR_EL2 = 0x0000000000000000

(lldb) x/6i \$pc

-> 0x402002f0: 0xd10043ff unknown sub sp, sp, #0x10

```
(lldb) si
```

Process 1 stopped

```
frame #0: 0x000000000000000200
```

0x204: udf #0x0

0x20c: udf #0x0

```
SPSR_EL2 = 0x0000000001003c9
```

```
(lldb) reg read SPSR EL2 ESR EL2 --format bin
```

[illegible][illegible]

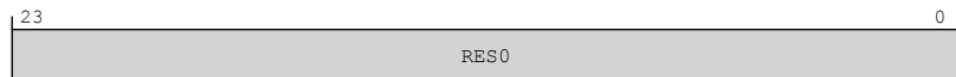
(lldb)

这里给出的错误是“非法执行状态”，进一步查看 ISS / ISS2 (Instruction Specific Syndrome)

ISS encoding for an exception from an Illegal Execution state, or a PC or SP alignment fault



ISS2 encoding for all other exceptions



在当前错误中 ISS/ISS2 都是保留位，无可利用信息。现在能做的就是先对 ISS 中给出的三种可能性进行筛选，进一步确定方向。PC or SP alignment 引发的异常我觉得可以先排除，上文提到过对于 EL1 stack 进行过反复验证，而 PC 指向的 `sub sp, sp, #0x10` 也不会存在什么架构对齐问题。

那么剩下的就是这个 **Illegal Execution state** 了，而 ESR_EL2 无法给出更多的信息了，我接着查看了 SPSR_EL2 的 IL (20bit)（注意：这里的 IL 和 ESR_EL2 的 IL，虽然缩写都是一样的，但意思完全不同，SPSR_EL2 的 IL 是指 Illegal Execution state，ESR_EL2 的 IL 是指 Instruction Length）：

IL, bit [20]

Illegal Execution state. Set to the value of PSTATE.IL on taking an exception to EL2, and copied to PSTATE.IL on executing an exception return operation in EL2.

The reset behavior of this field is:

- On a Warm reset, this field resets to an architecturally UNKNOWN value.

这里也没有过多的说明，这个 bit 应该是从 ESR_EL2 复制过来的。

线索到这里好像断掉了，不过可以肯定的是，由于导致了 **Illegal Execution state** 错误，将原本已经要在 EL1 执行的 `el1_entry` 给生生弹回到了 EL2，即在 `el1_entry` 入口处执行 `si` 命令后返回到了 EL2，这也能够解释为什么 `eret` 执行后能够看到正确的 `el1_entry` 地址和 PC 指向且 SPSR_EL2 / ESR_EL2 都正常。也就是说处理器的行为是，当执行 `eret` 指令时先检查 EL2_SP / ELR_EL2 / SPSR_EL2 配置，如果正常根据其指定跳到 EL1 entry 处，当要执行指令时再进行状态检查，这应该是一种 lazy check 机制，否则无法解释。

抱着大胆假设，小心求证的原则，接下来以 **Illegal Execution state** 作为关键字对 1 万 6 千多页的手册进行搜索，找到了 91 处，在 G1.15.5 节看到了如下描述：

G1.15.5 The Illegal Execution state exception

When the value of the [PSTATE.IL](#) bit is 1, any attempt to execute an instruction generates an Illegal Execution state exception. In AArch32 state, the [PSTATE.IL](#) bit can be set to 1 by one of the following:

- An illegal return, as described in [Illegal return events from AArch32 state](#).
- An illegal change to [PSTATE.M](#), as described in [Illegal changes to PSTATE.M](#).
- A legal return that sets [PSTATE.IL](#) to 1, as described in [Legal returns that set PSTATE.IL to 1](#).

An Illegal Execution state exception is taken in the same way as an Undefined Instruction exception in the current Exception level. If the current Exception level is EL2 using AArch32 state, the [HSR](#) provides additional syndrome information for the exception, see [Use of the HSR](#).

An Illegal Execution state exception has priority over any other Undefined Instruction exception that might arise from instruction execution.

Note

This section only describes the handling of an [Illegal Execution state](#) exception that is taken to an Exception level that is using AArch32 state.

On taking any exception to an Exception level that is using AArch32 state:

1. The value of the [PSTATE.IL](#) bit is 1 and this is copied to the [SPSR.IL](#) bit for the PE mode to which the exception is taken.
2. The [PSTATE.IL](#) bit is cleared to 0.

Note

This means that it is not possible for software to observe the value of [PSTATE.IL](#).

以上描述解答了为什么在 EL1 entry 处执行 `sub sp, sp, #0x10` 这样正常的指令都会产生异常并跳转到 `udf` 指令，因为当 `IL` bit 置位时，无论任何指令都会引发 **Illegal Execution state**，看来这是一种架构规定，严格来说不是指令引发的异常而是当处理器处于某种状态下不能有任何动作，任意动作都会抛出异常。这解答了其中一个巨大的疑问。那么另一个疑问是它到底是什么，由谁引发的？

接下来继续搜索，在手册的第 15804 页找到了 **J1.1.3.36 AArch64.ExceptionReturn** 伪码，他给出了在 AArch64 下所有异常返回的可能性，其中就包含了 **Illegal Execution state** 相关伪码如下：

```
// Attempts to change to an illegal state will invoke the Illegal Execution state mechanism
constant bits(2) source_el = PSTATE.EL;
constant boolean illegal_psr_state = IllegalExceptionReturn(spsr);
SetPSTATEFromPSR(spsr, illegal_psr_state);
ClearExclusiveLocal(ProcessorID());
SendEventLocal();
```

可以看到当执行了 **IllegalExceptionReturn** 后会根据一个 `illegal_psr_state` 来设置 `SPSR_ELn` 寄存器，代入到当前上下文中，这里就是设置 **Illegal Execution state** 的地方，接下来在手册 16375 页定位到 **J1.3.3.448 IllegalExceptionReturn** 伪码描述。

接下来对它进行详细的分析：

```

boolean IllegalExceptionReturn(bits(N) spsr)
// Check for illegal return:
// * To an unimplemented Exception level.
// * To EL2 in Secure state, when SecureEL2 is not enabled.
// * To EL0 using AArch64 state, with SPSR.M<0>==1.
// * To AArch64 state with SPSR.M<1>==1.
// * To AArch32 state with an illegal value of SPSR.M.
(valid, target) = ELFromSPSR(spsr);
if !valid then return TRUE;

// Check for return to higher Exception level.
if UInt(target) > UInt(PSTATE.EL) then return TRUE;

spsr_mode_is_aarch32 = (spsr<4> == '1');

// Check for illegal return:
// * To EL1, EL2 or EL3 with register width specified in the SPSR different from the
//   Execution state used in the Exception level being returned to, as determined by
//   the SCR_EL3.RW or HCR_EL2.RW bits, or as configured from reset.
// * To EL0 using AArch64 state when EL1 is using AArch32 state as determined by the
//   SCR_EL3.RW or HCR_EL2.RW bits or as configured from reset.
// * To AArch64 state from AArch32 state (should be caught by above).

```

我当前环境是要切换到 EL1 且 SPSR_EL2 配置是正常的，所以伪码的 ELFromSPSR 不会产生错误的，下面的 ELn 级别检查也不会有问题，我是从 (EL2->EL1) 不存在切换到更高的 ELn 级别。后续的 SPSR_EL2 的 M[4]我明确指明了是 AArch64 即 M[4]为 0，也没有任何问题。

当解决了这个 BUG 后，我把当前环境和代码再次给到 GPT 5.2，他给出的回答是，由于我的环境是 qemu 直接裸机启动到 EL2，没有 EL3 的实现，也就没有 ATF (ARM Trusted Firmware) 实现，所以 EL2.RW 位默认没有设置，而当真实环境与架构相关寄存器位本应该由固件设置的，所以才会碰到这样的问题...也算是个回答，姑且先放置在这里，待以后验证。

```
// Check for illegal return:
// * To EL1, EL2 or EL3 with register width specified in the SPSR different from the
//   Execution state used in the Exception level being returned to, as determined by
//   the SCR_EL3.RW or HCR_EL2.RW bits, or as configured from reset.
// * To EL0 using AArch64 state when EL1 is using AArch32 state as determined by the
//   SCR_EL3.RW or HCR_EL2.RW bits or as configured from reset.
// * To AArch64 state from AArch32 state (should be caught by above).
```

Copyright © 2013-2025 Arm Limited or its affiliates. All rights reserved.
Non-Confidential

idocode

```
(known, target_el_is_aarch32) = ELUsingAArch32K(target);
assert known || (target == EL0 && !ELUsingAArch32(EL1));
if known && spsr_mode_is_aarch32 != target_el_is_aarch32 then return TRUE;

// Check for illegal return from AArch32 to AArch64.
if UsingAArch32() && !spsr_mode_is_aarch32 then return TRUE;
// Check for illegal return to EL1 when HCR_EL2.TGE is set and when either of
// * SecureEL2 is enabled.
// * SecureEL2 is not enabled and EL1 is in Non-secure state.
if EL2Enabled() && target == EL1 && HCR_EL2.TGE == '1' then
    if (!IsSecureBelowEL3() || IsSecureEL2Enabled()) then return TRUE;

if (IsFeatureImplemented(FEAT_GCS) && PSTATE.EXLOCK == '0' &&
    PSTATE.EL == target && GetCurrentEXLOCKEN()) then
    return TRUE;

return FALSE;
```

查看 ELUsingAArch32K 实现

```
(boolean,boolean) ELUsingAArch32K(bits(2) el)
    return ELStateUsingAArch32K(el, IsSecureBelowEL3());
```

最终调用的是 ELStateUsingAArch32K


```

(boolean, boolean) ELStateUsingAArch32K(bits(2) el, boolean secure)
    assert HaveEL(el);

    if !HaveAArch32EL(el) then
        return (TRUE, FALSE); // Exception level is using AArch64
    elsif secure && el == EL2 then
        return (TRUE, FALSE); // Secure EL2 is using AArch64
    elsif !HaveAArch64() then
        return (TRUE, TRUE); // Highest Exception level, therefore all levels are using AArch64.

    // Remainder of function deals with the interprocessing cases when highest
    // Exception level is using AArch64.

    if el == EL3 then
        return (TRUE, FALSE);

    if (HaveEL(EL3) && SCR_EL3.RW == '0' &&
        (!secure || !IsFeatureImplemented(FEAT_SEL2) || SCR_EL3.EEL2 == '0')) then
        // AArch32 below EL3.
        return (TRUE, TRUE);

    if el == EL2 then
        return (TRUE, FALSE);

    if (HaveEL(EL2) && !ELIsInHost(EL0) && HCR_EL2.RW == '0' &&
        (!secure || (IsFeatureImplemented(FEAT_SEL2) && SCR_EL3.EEL2 == '1')))) then
        // AArch32 below EL2.
        return (TRUE, TRUE);

    if el == EL1 then
        return (TRUE, FALSE);

```

前几个判断我环境都不满足，我的 EL2 不是 Secure EL2 且目标是 EL1；当前环境肯定是支持 AArch64 的；由于我是 qemu 跳过 ATF 直接以 EL2 裸机运行的所以根本就不存在 EL3；由于没有 EL3 也就谈不上 EL3.RW 位了。当我看到标红框部分的伪码时，顺便查了一下当前环境的 HCR_EL2

```

(lldb) reg read HCR_EL2
HCR_EL2 = 0x0000000000000001
(lldb) |

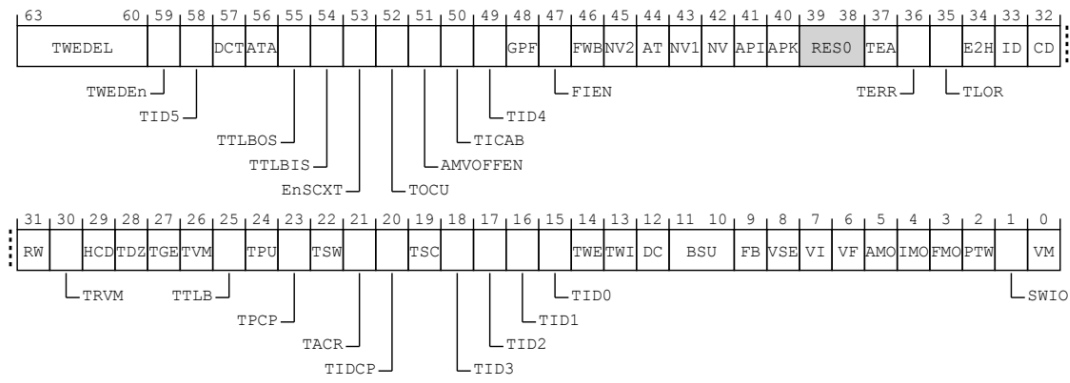
```

设置 HCR_EL2:

对于这个寄存器，更多关注的是它的 VM / TGE / TFB / TWI 相关 bit。VM bit = 1 就是开启 Stage-2 的地址转换机制。当 EL1(MMU 使能) 完成 GVA->IPA 转换后，如果 VM bit = 1 且 VTTBR_EL2 设置了有效页表，则进行最终的 IPA->PA 转换；如果 VTTBR_EL2 = 0 即便 VM bit = 1 也不会做 IPA->PA 转换，此时 IPA = PA。所以在

我当前环境 VM bit 是什么不重要，由于还没有设置 VTTBR_EL2，所以不会有什么事情发生。（GPT 5.2 与 Claude Sonnet 4.5 对这个问题的解答显然是错误的）

HCR_EL2 寄存器：



我的代码开始至设置了 VM bit 位，对于其它位直接使用系统默认值，通过以上观察发现，RW bit 位也是 0，看看 RW bit 的说明：

RW	Meaning
0b0	Lower levels are all AArch32.
0b1	The Execution state for EL1 is AArch64. The Execution state for EL0 is determined by the current value of PSTATE.nRW when executing at EL0.

In an implementation that includes EL3, when EL2 is not enabled in Secure state, the PE behaves as if this bit has the same value as the [SCR_EL3.RW](#) bit for all purposes other than a direct read or write access of HCR_EL2.

The RW bit is permitted to be cached in a TLB.

When the Effective value of HCR_EL2.{E2H, TGE} is {1, 1}, the Effective value of this field is 1.

The reset behavior of this field is:

- On a Warm reset, this field resets to an architecturally UNKNOWN value.

Otherwise:

Reserved, RAO/WI.

其实到这里已经定位到了 BUG 产生的原因了。我结合当前运行环境，又仔细看了下 eret 指令的说明：

C6.2.157 ERET

Exception return

This instruction restores [PSTATE](#) from the SPSR, and branches to the address held in the ELR.

The SPSR is checked for the current Exception level for an illegal return event. See [Illegal exception returns from AArch64 state](#).

ERET is UNDEFINED at EL0.

ERET

```
constant boolean pac = FALSE;
```

```
if PSTATE.EL == EL0 then UNDEFINED;
```

```
AArch64_ExceptionReturn(target, SPSPR_ElX[1]);
```

这里已经给出了操作步骤，其中明确给出了上文提到的伪码 **J1.1.3.36 AArch64.ExceptionReturn**。并且在 `eret` 指令说明中也给出了 **D1.4.4.2 Illegal exception returns from AArch64 state** 的相关状态，其中当 `SPSR_EL2` 的 `M[4]` 为 0 的时候明确了以下的可能性（注意：以下是或的状态）：

- $M[1]$ is 1.

- M[3:0] are 0b0001.
- The Exception level being returned to is using AArch32 state, as determined by SCR_EL3.RW, HCR_EL2.RW, or as configured from reset.

HCR EL2 =

[illegible]

* thread #1, stop reason = instruction step into

```
frame #0: 0x00000000402002f0 hyper-vmx`el1_entry at main.rs:180
```

177

```
178 #[inline(never)]
```

179 #[no mangle]

```
180 extern "C" fn el1 entrv() -> !
```

181 {

```
182     let el = read current el():
```

183

(lldb) si

Process 1 stopped

* thread #1, stop reason = instruction step into

frame #0: 0x00000000402002f4 hyper-vmm`el1_entry at main.rs:182:14

179 #[no_mangle]

180 extern "C" fn el1_entry() -> !

181 {

-> 182 let _el = read_current_el();

183

184 unsafe {

185 loop {

(lldb) x/6i \$pc

-> 0x40200304: 0xf90003e8 unknown str x8, [sp]

0x40200308: 0x14000001 unknown b

0x4020030c ; <+28> at main.rs:186:13

0x4020030c: 0xd2800840 unknown mov x0,

#0x42 ; =66

0x40200310: 0xd503201f unknown nop

0x40200314: 0x17fffffe unknown b 0x4020030c ;

<+28> at main.rs:186:13

0x40200318: 0xd102c3ff unknown sub sp, sp, #0xb0

(lldb) si

(lldb) memory read \$sp

0x40202ff0: 01 00 00 00 00 00 00 00 04 00 00 00 00 00 00 00

0x40203000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

现在回想起来，当初跟踪代码的时候已经能够跟踪到 el1_entry，而此时查看 EL1 所有相关寄存器状态和 EL1 stack 都正常，**就被 AI 误导，以为当前环境已经是 EL1（即便现在事后来看也很难不被误导）**，此时没有再把 eret 和 EL2 作为排查目标。